Name: M.B.S.S.V.Avinash
Roll No: 26000957 | Branch: CSM | Section: 2

PYTHON PROGRAMMING PROJECTS

Beginner-Friendly Project Report

This document organizes the supplied programming topics into a consistent academic project format. Each project includes a problem statement, objectives, assumptions, algorithm, simple flowchart, pseudocode, Python source code, test cases, sample output, and conclusion.

Topics included

- University Scholarship Allocation
- Railway Fare Calculator
- Course Admission System
- Grace Marks and Student Classification
- College Timetable Conflict Checker
- Environmental Monitoring Readings
- Mobile Plan Comparison
- Public Transport Revenue Reconciliation
- Hospital Billing and Concessions
- Warehouse Inventory Reconciliation

Prepared for learning and practice

PROJECT 1: UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement

A university has a limited number of scholarships. Applicants must satisfy all eligibility rules, then eligible students are ranked fairly. The top three receive scholarships and the rest are waitlisted.

2. Objectives

- Calculate average marks.
- Check aggregate, subject minimum, attendance and income.
- Rank eligible students using fixed tie-break rules.
- Show awards, waitlist and rejection reasons.

3. Assumptions

- Aggregate must be at least 60%.
- Every subject mark must be at least 40.
- Attendance must be at least 75%.
- Annual family income must not exceed I3,00,000.
- Three scholarships are available.

4. Algorithm

- Set the rule constants.
- Read applicant records.
- Calculate aggregate and check every eligibility rule.
- Save eligible and rejected applicants.
- Sort eligible applicants by aggregate descending, Maths descending, income ascending, attendance descending, then name.
- Award the first three and waitlist the rest.
- Print results and rejection reasons.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
aggregate = sum(marks) / len(marks)
IF aggregate < 60: reject
IF any mark < 40: reject
IF attendance < 75: reject
IF income > 300000: reject
Otherwise eligible
Sort by (-aggregate, -maths, income, -attendance, name)
First 3 = AWARDED; others = WAITLIST
```

7. Python Source Code

```
SEATS = 3
MIN_AGG = 60
MIN_SUBJECT = 40
MIN_ATTENDANCE = 75
MAX_INCOME = 300000
applicants = [
    {"name": "Asha", "marks": [85, 88, 82], "attendance": 90, "income": 150000},
    {"name": "Bharat", "marks": [85, 90, 80], "attendance": 92, "income": 140000},
    {"name": "Charan", "marks": [90, 95, 85], "attendance": 88, "income": 200000},
    {"name": "Divya", "marks": [85, 90, 80], "attendance": 85, "income": 100000},
```

```
    {"name": "Esha", "marks": [95, 95, 35], "attendance": 95, "income": 100000},
]

eligible = []
rejected = []

for student in applicants:
    avg = sum(student["marks"]) / len(student["marks"])
    student["aggregate"] = avg
    reasons = []
    if avg < MIN_AGG:
        reasons.append("aggregate below 60%")
    if any(mark < MIN_SUBJECT for mark in student["marks"]):
        reasons.append("a subject mark is below 40")
    if student["attendance"] < MIN_ATTENDANCE:
        reasons.append("attendance below 75%")
    if student["income"] > MAX_INCOME:
        reasons.append("income above Rs 3,00,000")
    if reasons:
        rejected.append((student["name"], ", ".join(reasons)))
    else:
        eligible.append(student)

eligible.sort(key=lambda s: (-s["aggregate"], -s["marks"][0],
                             s["income"], -s["attendance"], s["name"]))

print("RANK NAME   AGGREGATE STATUS")
for rank, student in enumerate(eligible, start=1):
    status = "AWARDED" if rank <= SEATS else "WAITLIST"
    print(rank, student["name"], f'{student["aggregate"]:.2f}%', status)

print("\nREJECTED APPLICANTS")
for name, reason in rejected:
    print(name, "-", reason)
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Esha has marks 95, 95, 35	Rejected: a subject mark is below 40
2	Attendance is 70%	Rejected: attendance below 75%
3	Income is I5,00,000	Rejected: income above limit
4	Aggregate is 55%	Rejected: aggregate below 60%
5	Five eligible applicants, three seats	First three awarded; remaining waitlisted

9. Sample Output

```
RANK NAME   AGGREGATE STATUS
1 Charan 90.00% AWARDED
2 Bharat 85.00% AWARDED
3 Divya 85.00% AWARDED
4 Asha 85.00% WAITLIST

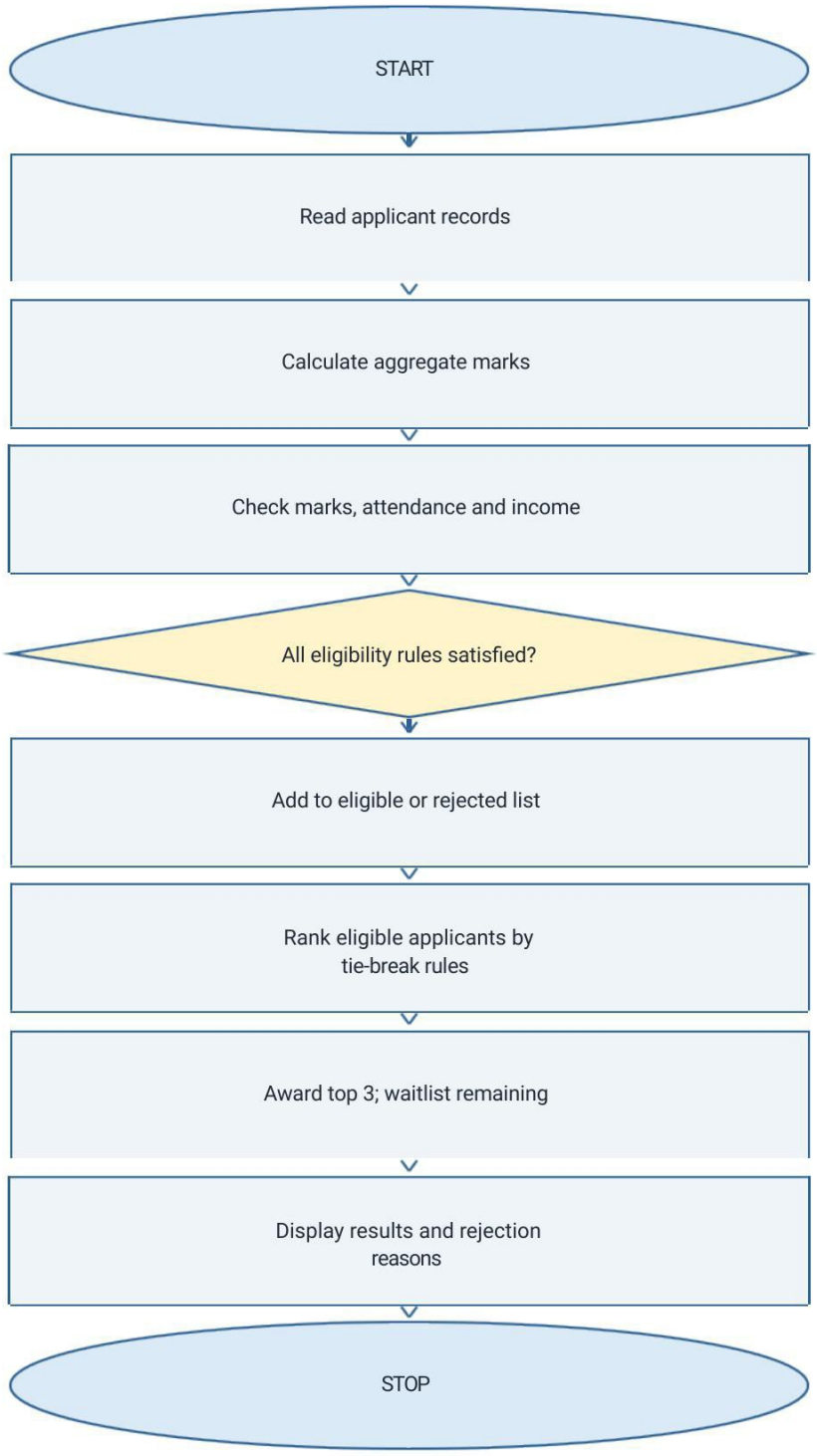
REJECTED APPLICANTS
Esha - a subject mark is below 40
```

10. Conclusion

The system checks all eligibility rules, ranks only eligible applicants and clearly explains rejections. Fixed tie-break rules make the selection repeatable.

PROJECT 1: UNIVERSITY SCHOLARSHIP ALLOCATION

Flowchart



PROJECT 2: RAILWAY FARE CALCULATOR

1. Problem Statement

A railway counter needs to calculate fares for groups travelling different distances and classes. Age-based concessions, reservation fees and cancellation refunds must be included.

2. Objectives

- Calculate distance-slab fare.
- Apply class multiplier and passenger concession.
- Add reservation fees.
- Calculate cancellation penalty and refund.

3. Assumptions

- Rates: first 100 km I1/km, next 200 km I0.80/km, above 300 km I0.60/km.
- Minimum base fare is I30.
- SL: multiplier 1.0, fee I20, clerkage I120.
- 3A: multiplier 2.2, fee I40, clerkage I180.
- 2A: multiplier 3.0, fee I50, clerkage I200.
- 1A: multiplier 4.5, fee I60, clerkage I240.
- Under 5 travels free without a berth.

4. Algorithm

- Calculate base fare by distance slabs.
- Multiply by class rate.
- Apply the passenger's concession.
- Add reservation fee except for infants.
- Add passenger totals.
- If cancelling, calculate penalty from hours before departure and show refund.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
base = slab fare, minimum 30
class_fare = base * class_multiplier
fare = class_fare * (1 - concession_percent / 100)
reservation = 0 for infants, otherwise class fee
total = fare + reservation
Penalty: >=48h clerkage; >=12h 25%; >=4h 50%; otherwise 100%
Refund = max(0, fare - penalty)
```

7. Python Source Code

```
SLABS = [(100, 1.00), (300, 0.80), (float("inf"), 0.60)]
CLASSES = {
    "SL": (1.0, 20, 120),
    "3A": (2.2, 40, 180),
    "2A": (3.0, 50, 200),
    "1A": (4.5, 60, 240)
}
def base_fare(distance):
    fare = 0
    previous = 0
```

```
    for limit, rate in SLABS:
        if distance > previous:
            km = min(distance, limit) - previous
            fare += km * rate
        previous = limit
        if distance <= limit:
            break
    return max(fare, 30)
def concession(age, gender):
    if age < 5:
        return 100
    if age <= 11:
        return 50
    if gender == "F" and age >= 58:
        return 50
    if gender == "M" and age >= 60:
        return 40
    return 0
def passenger_fare(age, gender, class_code, distance):
    multiplier, fee, clerkage = CLASSES[class_code]
    class_fare = base_fare(distance) * multiplier
    percent = concession(age, gender)
    fare = class_fare * (1 - percent / 100)
    reservation = 0 if percent == 100 else fee
    return fare, reservation, fare + reservation
def cancellation(fare, class_code, hours):
    clerkage = CLASSES[class_code][2]
    if hours >= 48:
        penalty = min(clerkage, fare)
    elif hours >= 12:
        penalty = fare * 0.25
    elif hours >= 4:
        penalty = fare * 0.50
    else:
        penalty = fare
    return penalty, max(0, fare - penalty)
distance = 450
class_code = "3A"
passengers = [("Adult", 35, "M"), ("Child", 8, "F"), ("Infant", 2, "M")]
grand_total = 0
print("Passenger Fare Reservation Total")
for name, age, gender in passengers:
    fare, fee, total = passenger_fare(age, gender, class_code, distance)
    grand_total += total
    print(name, f"{fare:.2f}", f"{fee:.2f}", f"{total:.2f}")
print("Grand total:", f"{grand_total:.2f}")
penalty, refund = cancellation(770, "3A", 72)
print("Cancellation penalty:", f"{penalty:.2f}")
print("Refund:", f"{refund:.2f}")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Distance 1 km	Base fare I30 minimum
2	Distance 50 km	Base fare I50
3	Distance 101 km	Base fare I100.80
4	3A, 450 km, adult	Fare I770 + I40 reservation = I810
5	Infant age 2	Total I0
6	Cancel 2 hours before	No refund

9. Sample Output

```
Passenger Fare  Reservation Total
```

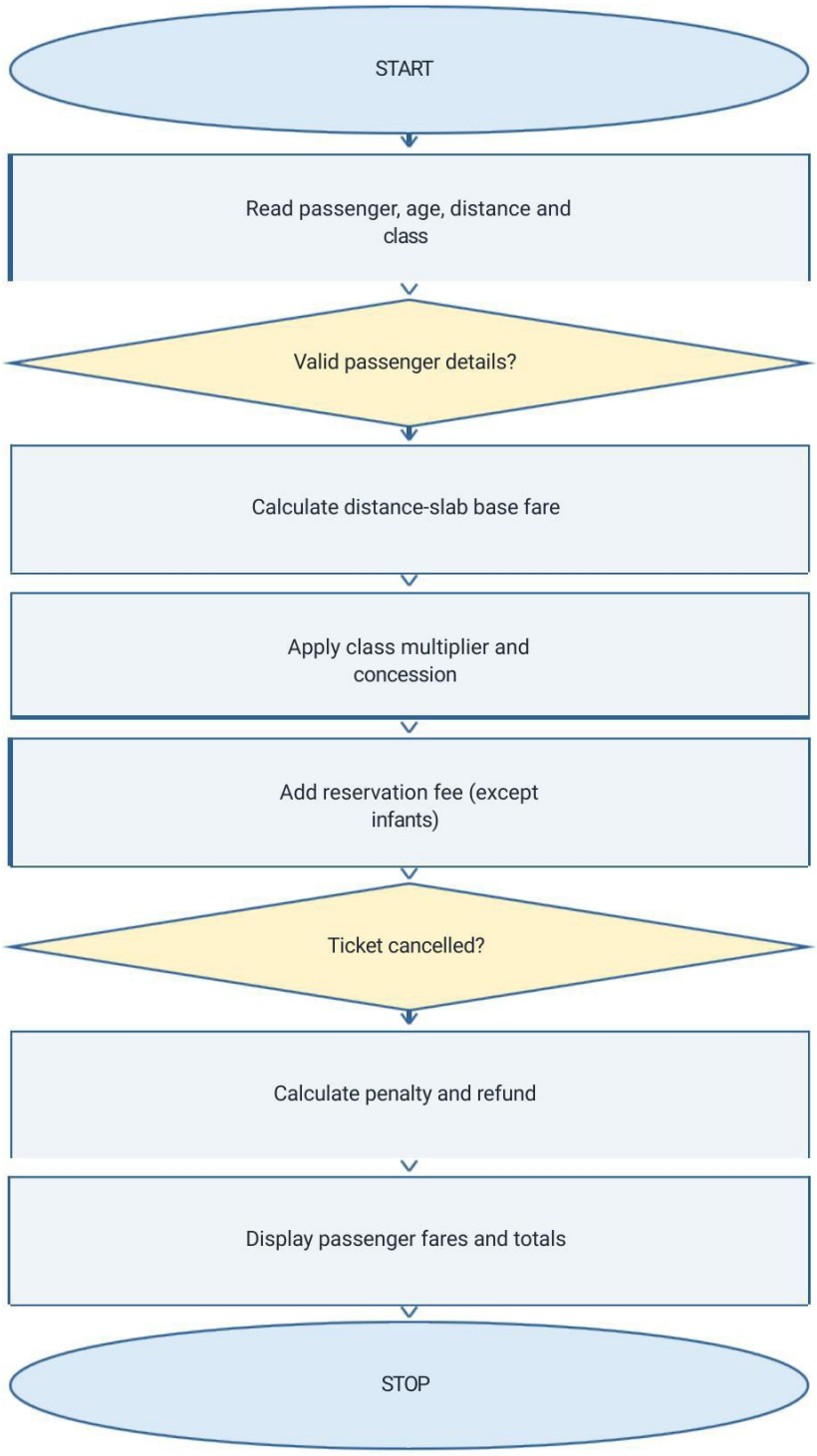
Adult 770.00 40.00 810.00
Child 385.00 40.00 425.00
Infant 0.00 0.00 0.00
Grand total: 1235.00
Cancellation penalty: 180.00
Refund: 590.00

10. Conclusion

The calculator combines distance slabs, class multipliers, concessions and cancellation rules. Each passenger is calculated separately, making it suitable for mixed-age groups.

PROJECT 2: RAILWAY FARE CALCULATOR

Flowchart



PROJECT 3: COURSE ADMISSION SYSTEM

1. Problem Statement

A college has multiple courses with different subject requirements, cutoffs and seat limits. Candidates are checked against the selected course, ranked, and admitted or waitlisted.

2. Objectives

- Store course-specific rules.
- Check required subjects and minimum marks.
- Apply aggregate cutoffs.
- Rank candidates course by course.
- Report admissions, waitlists and rejection reasons.

3. Assumptions

- CSE: Maths 60, Physics 50, English 40; cutoff 70%; 3 seats.
- PHY: Maths 50, Physics 60; cutoff 60%; 2 seats.
- COM: Accountancy 50, English 40; cutoff 55%; 2 seats.
- Ranking: aggregate descending, tie-break subject descending, entrance score descending, older date of birth first, then name.

4. Algorithm

- Read each candidate and course.
- Reject unknown courses.
- Check required subjects and marks.
- Calculate aggregate and compare with cutoff.
- Sort eligible candidates per course.
- Mark candidates ADMITTED or WAITLIST.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
For each candidate:
  Find course rules
  Check required subjects and minimum marks
  Check aggregate cutoff
  If eligible, add to course list
For each course:
  Sort by aggregate, primary subject, entrance score,
  date of birth, and name
  Admit up to seat limit; waitlist the rest
```

7. Python Source Code

```
COURSES = {
    "CSE": ({"Maths": 60, "Physics": 50, "English": 40}, 70, 3, "Maths"),
    "PHY": ({"Maths": 50, "Physics": 60}, 60, 2, "Physics"),
    "COM": ({"Accountancy": 50, "English": 40}, 55, 2, "Accountancy")
}
candidates = [
    {"name": "Arun", "course": "CSE",
     "marks": {"Maths": 85, "Physics": 75, "English": 80},
     "entrance": 88, "dob": "2006-03-10"},
    {"name": "Bhavna", "course": "CSE",
```

```
        "marks": {"Maths": 85, "Physics": 75, "English": 80},
        "entrance": 92, "dob": "2006-01-10"},
    {"name": "Chetan", "course": "CSE",
        "marks": {"Maths": 70, "Physics": 90, "English": 80},
        "entrance": 90, "dob": "2005-10-01"},
    {"name": "Farah", "course": "CSE",
        "marks": {"Maths": 75, "Physics": 60, "English": 60},
        "entrance": 85, "dob": "2006-05-01"}
]
eligible = {code: [] for code in COURSES}
rejected = []
for c in candidates:
    if c["course"] not in COURSES:
        rejected.append((c["name"], "Unknown course"))
        continue
    required, cutoff, seats, primary = COURSES[c["course"]]
    reasons = []
    for subject, minimum in required.items():
        if subject not in c["marks"]:
            reasons.append(subject + " missing")
        elif c["marks"][subject] < minimum:
            reasons.append(subject + " below minimum")
    avg = sum(c["marks"].values()) / len(c["marks"])
    c["aggregate"] = avg
    if avg < cutoff:
        reasons.append("aggregate below cutoff")
    if reasons:
        rejected.append((c["name"], ", ".join(reasons)))
    else:
        eligible[c["course"]].append(c)
for code, group in eligible.items():
    required, cutoff, seats, primary = COURSES[code]
    group.sort(key=lambda c: (-c["aggregate"], -c["marks"][primary],
                              -c["entrance"], c["dob"], c["name"]))
    print("\nCourse:", code)
    for rank, c in enumerate(group, 1):
        status = "ADMITTED" if rank <= seats else "WAITLIST"
        print(rank, c["name"], f'{c["aggregate"]:.2f}%', status)
print("\nRejected:", rejected)
```

8. Test Cases

Test	Input / Condition	Expected Result
1	CSE candidate Maths 55	Rejected: Maths below minimum
2	Required Physics mark missing	Rejected: Physics missing
3	CSE aggregate 65%	Rejected: aggregate below cutoff
4	Candidate applies for MBA	Invalid application: unknown course
5	Only one eligible candidate for two seats	Candidate admitted; no waitlist

9. Sample Output

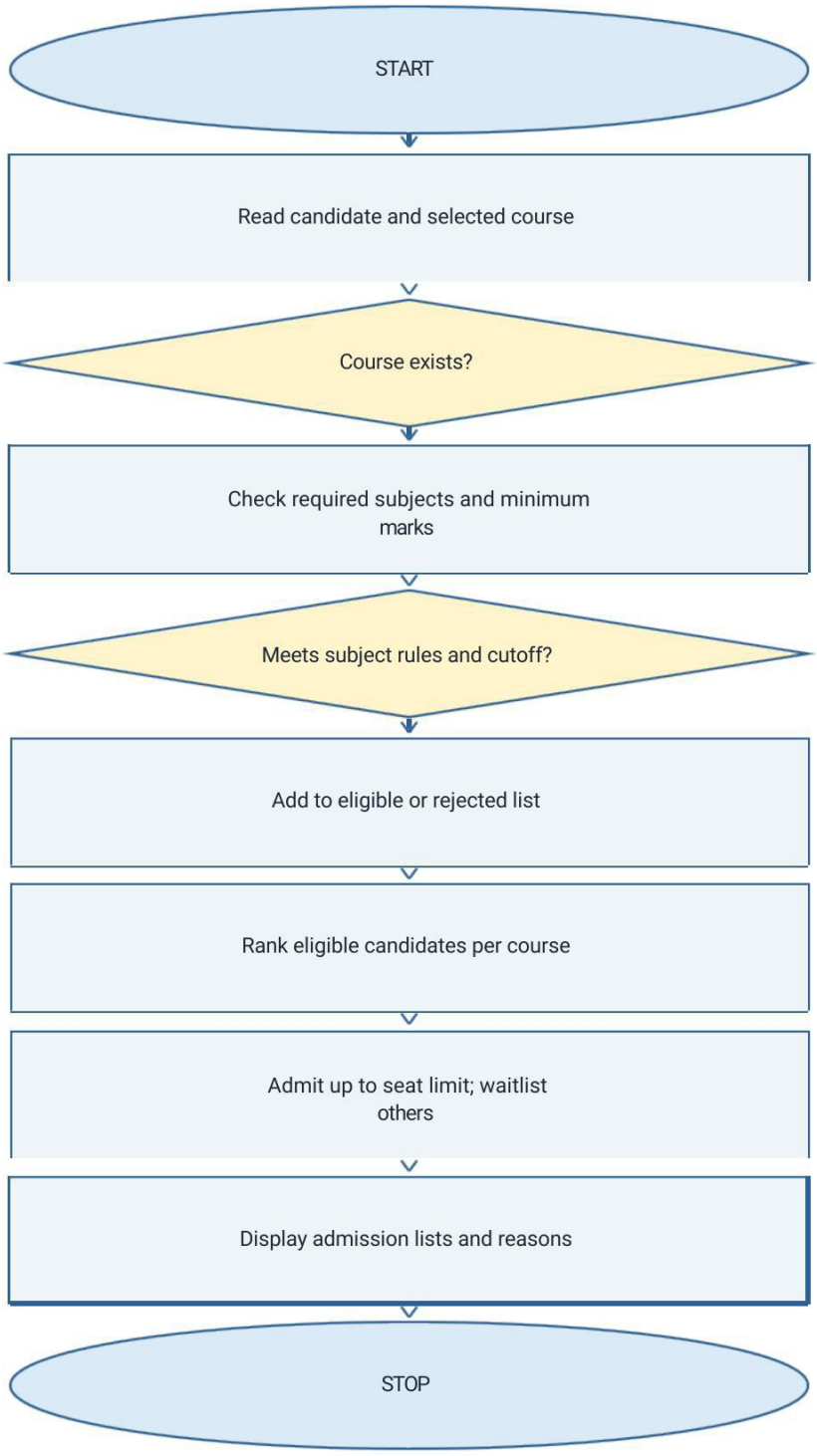
```
Course: CSE
1 Bhavna 80.00% ADMITTED
2 Arun 80.00% ADMITTED
3 Chetan 80.00% ADMITTED
Rejected: [('Farah', 'Physics below minimum, English below minimum')]
```

10. Conclusion

The admission engine applies each course's own rules and ranks eligible candidates using documented tie-breakers. Rejection reasons help make the process transparent.

PROJECT 3: COURSE ADMISSION SYSTEM

Flowchart



PROJECT 4: GRACE MARKS AND STUDENT CLASSIFICATION

1. Problem Statement

Students must pass every subject. Limited grace marks may help students close to the pass mark, but grace must stay within per-subject and per-student limits and marks must never exceed 100.

2. Objectives

- Validate marks.
- Apply pass grace only under policy limits.
- Apply class-upgrade grace where allowed.
- Calculate final total, percentage and class.

3. Assumptions

- Pass mark 40; maximum mark 100.
- A failed subject can receive grace only when short by at most 5.
- At most 2 failed subjects.
- Maximum grace per subject 5; per student 10.
- Class grace shortfall at most 5 and must respect all limits.
- Classes: Distinction ≥75%, First ≥60%, Second ≥50%, Pass ≥40%, otherwise Fail.

4. Algorithm

- Validate every mark.
- Find failed subjects.
- If policy conditions fail, classify as FAIL.
- Raise eligible failed subjects to 40.
- Check whether class grace is allowed.
- Calculate final percentage and classification.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
Validate marks from 0 to 100
Find subjects below 40
If more than 2 failed or any needs >5 grace: FAIL
If total pass grace >10: FAIL
Raise failed marks to 40
If next class boundary is within 5 marks and limits allow:
  apply class grace to a suitable subject
Calculate final total and percentage
Classify by percentage
```

7. Python Source Code

```
PASS_MARK = 40
MAX_MARK = 100
MAX_FAILED = 2
MAX_SUBJECT_GRACE = 5
MAX_TOTAL_GRACE = 10
def classify(percent):
    if percent >= 75:
        return "Distinction"
```

```
    if percent >= 60:
        return "First Class"
    if percent >= 50:
        return "Second Class"
    if percent >= 40:
        return "Pass Class"
    return "Fail"
def process_marks(marks):
    if not marks or any(not isinstance(m, (int, float)) or m < 0 or m > 100 for m in marks):
        return "INVALID", marks, 0, 0, "Invalid marks"
    final = marks[:]
    grace = [0] * len(marks)
    failed = [i for i, mark in enumerate(marks) if mark < PASS_MARK]
    if len(failed) > MAX_FAILED:
        return "FAIL", final, 0, sum(final), "Too many failed subjects"
    for i in failed:
        need = PASS_MARK - final[i]
        if need > MAX_SUBJECT_GRACE:
            return "FAIL", final, 0, sum(final), "Grace limit exceeded"
        grace[i] = need
    if sum(grace) > MAX_TOTAL_GRACE:
        return "FAIL", final, 0, sum(final), "Total grace limit exceeded"
    for i in failed:
        final[i] += grace[i]
    total_grace = sum(grace)
    total = sum(final)
    percent = total / (MAX_MARK * len(final)) * 100
    # Class grace is considered only for a student who passed all subjects.
    boundaries = [50, 60, 75]
    next_boundary = next((b for b in boundaries if percent < b), None)
    if next_boundary is not None:
        target_total = next_boundary * len(final)
        shortfall = target_total - total
        if 0 < shortfall <= MAX_SUBJECT_GRACE and total_grace + shortfall <= MAX_TOTAL_GRACE:
            for i in sorted(range(len(final)), key=lambda j: final[j]):
                if final[i] + shortfall <= MAX_MARK and grace[i] + shortfall <= MAX_SUBJECT_GRACE:
                    final[i] += shortfall
                    grace[i] += shortfall
                    total += shortfall
                    total_grace += shortfall
                    break
    percent = total / (MAX_MARK * len(final)) * 100
    return "PASS", final, total_grace, total, classify(percent)
students = {
    "Priya": [80, 85, 78, 90, 72],
    "Tarun": [38, 39, 60, 70, 80],
    "Uma": [35, 35, 35, 70, 70],
    "Arjun": [105, 70, 70, 70, 70]
}
for name, marks in students.items():
    status, final, grace, total, result = process_marks(marks)
    print(name, status, "Final marks:", final,
          "Grace:", grace, "Total:", total, "Result:", result)
```

8. Test Cases

Test	Input / Condition	Expected Result
1	80, 85, 78, 90, 72	Pass, total 405, Distinction
2	38, 39, 60, 70, 80	Pass grace raises both failed subjects to 40
3	35, 35, 35, 70, 70	Fail: three failed subjects
4	30, 80, 80, 80, 80	Fail: first subject needs more than 5 grace
5	105, 70, 70, 70, 70	Invalid: mark exceeds 100

9. Sample Output

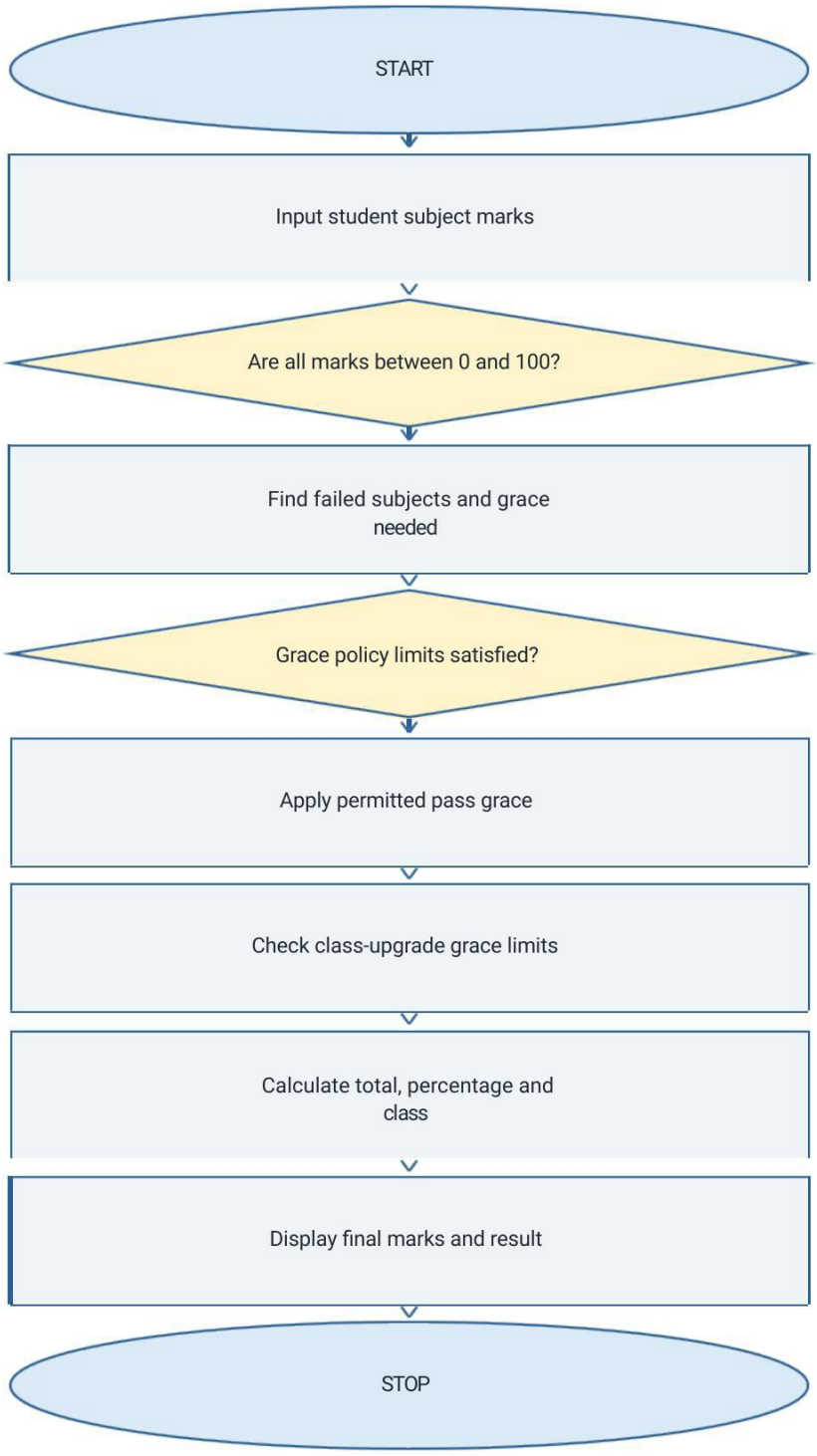
```
Priya PASS Final marks: [80, 85, 78, 90, 72] Grace: 0 Total: 405 Result: Distinction
Tarun PASS Final marks: [40, 40, 60, 70, 80] Grace: 3 Total: 290 Result: Second Class
Uma FAIL
Arjun INVALID
```

10. Conclusion

The program validates marks, applies grace only within defined limits and reports final marks and class. The grace rules are kept separate so the policy can be adjusted easily.

PROJECT 4: GRACE MARKS AND STUDENT CLASSIFICATION

Flowchart



PROJECT 5: COLLEGE TIMETABLE CONFLICT CHECKER

1. Problem Statement

A college timetable contains course, day, start time, end time and room. Overlapping periods and room double-bookings must be detected.

2. Objectives

- Represent classes as tuples.
- Convert times into minutes.
- Detect time overlaps.
- Find room conflicts and unavailable rooms.
- Validate timetable entries.

3. Assumptions

- Time format is HH:MM using a 24-hour clock.
- Days are Mon to Sat.
- Classes do not cross midnight.
- Back-to-back classes are allowed.
- Room names must exist in the supplied room list.

4. Algorithm

- Convert each time to minutes.
- Compare pairs on the same day.
- Two periods overlap if start1 < end2 and start2 < end1.
- Keep overlapping pairs using the same room as room conflicts.
- For a requested slot, list rooms used during any overlap.
- Validate data and report errors.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
time_to_minutes(t) = hours * 60 + minutes
overlap = start1 < end2 AND start2 < end1
Compare each pair on the same day
If overlap and room is same, report room conflict
Validate day, time order, and room
```

7. Python Source Code

```
def time_to_minutes(value):
    hour, minute = map(int, value.split(":"))
    if not (0 <= hour <= 23 and 0 <= minute <= 59):
        raise ValueError("Invalid time")
    return hour * 60 + minute
def overlaps(start1, end1, start2, end2):
    return start1 < end2 and start2 < end1
def find_overlaps(timetable):
    result = []
    for i in range(len(timetable)):
        for j in range(i + 1, len(timetable)):
            a, b = timetable[i], timetable[j]
            if a[1] == b[1]:
                if overlaps(time_to_minutes(a[2]), time_to_minutes(a[3]),
```

```
                                    time_to_minutes(b[2]), time_to_minutes(b[3])):
                            result.append((a, b))
    return result
def find_room_conflicts(timetable):
    return [(a, b) for a, b in find_overlaps(timetable) if a[4] == b[4]]
def unavailable_rooms(timetable, day, start, end):
    busy = set()
    s, e = time_to_minutes(start), time_to_minutes(end)
    for course, class_day, begin, finish, room in timetable:
        if class_day == day and overlaps(s, e, time_to_minutes(begin), time_to_minutes(finish)):
            busy.add(room)
    return busy
rooms = {"R101", "R102", "Lab1"}
timetable = [
    ("Maths", "Mon", "09:00", "10:00", "R101"),
    ("Physics", "Mon", "09:30", "10:30", "R101"),
    ("Chemistry", "Mon", "09:00", "10:00", "Lab1"),
    ("English", "Mon", "10:30", "11:30", "R102")
]
print("Overlapping pairs:")
for a, b in find_overlaps(timetable):
    print(a[0], "-", b[0])
print("Room conflicts:")
for a, b in find_room_conflicts(timetable):
    print(a[0], "and", b[0], "in", a[4])
print("Busy rooms:", sorted(unavailable_rooms(timetable, "Mon", "09:00", "10:00")))
```

8. Test Cases

Test	Input / Condition	Expected Result
1	09:00–10:00 and 09:30–11:00	Overlap True
2	09:00–10:00 and 10:00–11:00	Overlap False
3	Same time, different days	No overlap
4	Same time, same room	Room conflict found
5	Same time, different rooms	Overlap but no room conflict

9. Sample Output

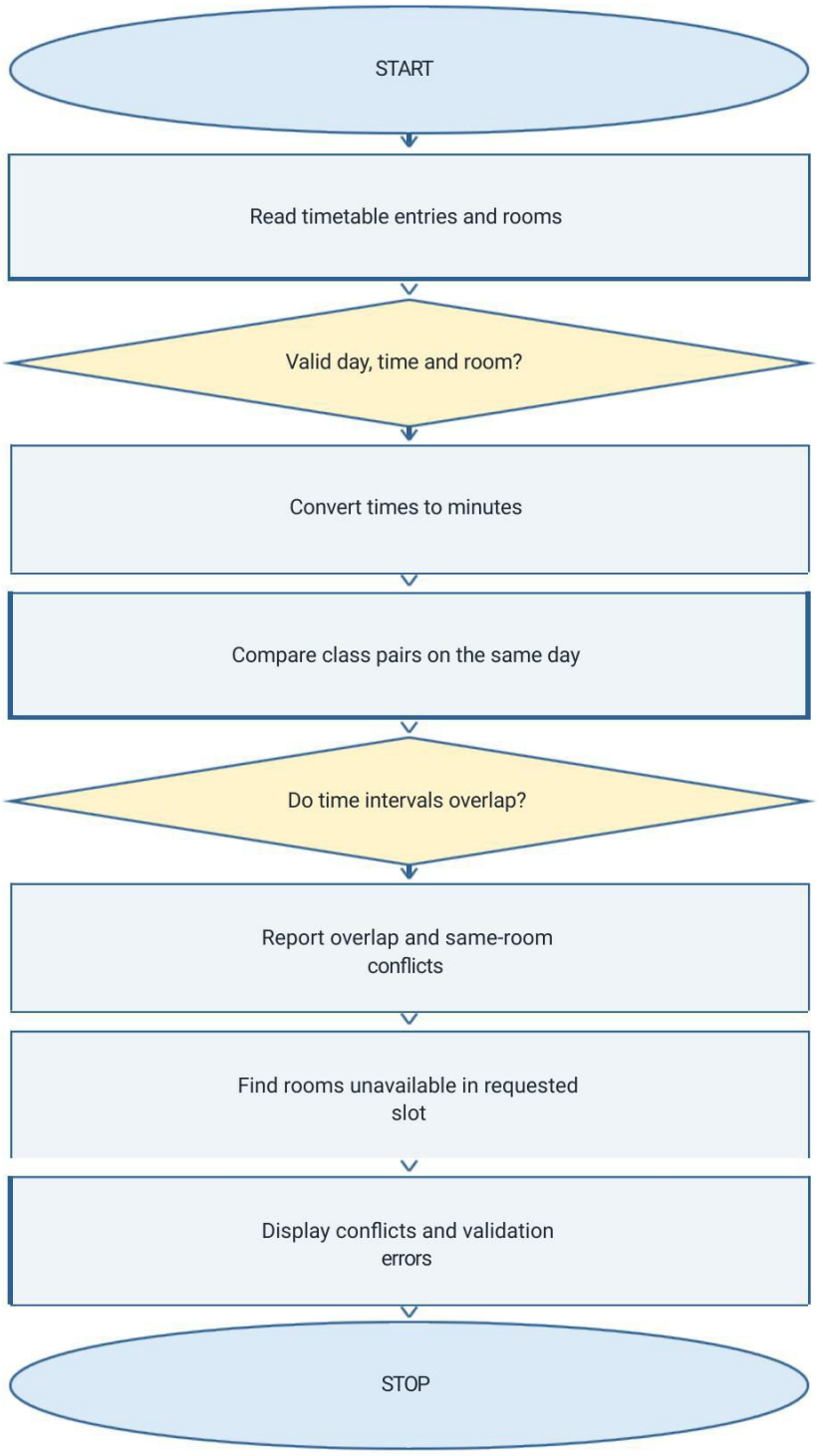
```
Overlapping pairs:
Maths - Physics
Maths - Chemistry
Physics - Chemistry
Room conflicts:
Maths and Physics in R101
Busy rooms: ['Lab1', 'R101']
```

10. Conclusion

The program uses simple time conversion and interval comparisons to identify overlapping classes, room conflicts and unavailable rooms.

PROJECT 5: COLLEGE TIMETABLE CONFLICT CHECKER

Flowchart



PROJECT 6: ENVIRONMENTAL MONITORING READINGS

1. Problem Statement

Monitoring stations collect measurements such as pollutant levels. Invalid readings must be ignored before averages, extremes and categories are calculated.

2. Objectives

- Store readings in nested lists.
- Ignore text and negative readings.
- Calculate average, minimum and maximum.
- Classify individual readings and station averages.

3. Assumptions

- A station is [name, readings].
- Valid readings are numeric and non-negative.
- Thresholds: up to 50 Good, up to 100 Moderate, up to 150 Unhealthy, above 150 Hazardous.
- A station without valid readings is reported as having no valid readings.

4. Algorithm

- Set thresholds.
- Clean each station's readings.
- Count invalid readings.
- Calculate average, minimum and maximum.
- Classify readings and average.
- Print a report.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
valid = numeric readings >= 0
if no valid readings: report no valid readings
average = sum(valid) / count(valid)
minimum = min(valid)
maximum = max(valid)
classify each value using ordered thresholds
```

7. Python Source Code

```
THRESHOLDS = [(50, "Good"), (100, "Moderate"),
                (150, "Unhealthy"), (float("inf"), "Hazardous")]
def clean_readings(readings):
    valid = []
    skipped = 0
    for value in readings:
        if isinstance(value, (int, float)) and value >= 0:
            valid.append(value)
        else:
            skipped += 1
    return valid, skipped
def classify(value):
    for limit, label in THRESHOLDS:
        if value <= limit:
            return label
stations = [
```

```
        ["Station A", [10, 20, 30, "error", -3]],
        ["Station B", [55, 75, 110, 160]],
        ["Station C", []]
]
for name, readings in stations:
        valid, skipped = clean_readings(readings)
        print("\nStation:", name)
        print("Skipped invalid readings:", skipped)
        if not valid:
                print("No valid readings")
                continue
        average = sum(valid) / len(valid)
        print("Valid readings:", valid)
        print("Average:", round(average, 2), "-", classify(average))
        print("Minimum:", min(valid))
        print("Maximum:", max(valid))
        print("Categories:", [classify(x) for x in valid])
```

8. Test Cases

Test	Input / Condition	Expected Result
1	[10, 20, 30]	Average 20.0
2	[5, 1, 9]	Minimum 1, maximum 9
3	50	Good
4	51	Moderate
5	[10, "x", -3, 0, 7.5]	Valid readings [10, 0, 7.5]

9. Sample Output

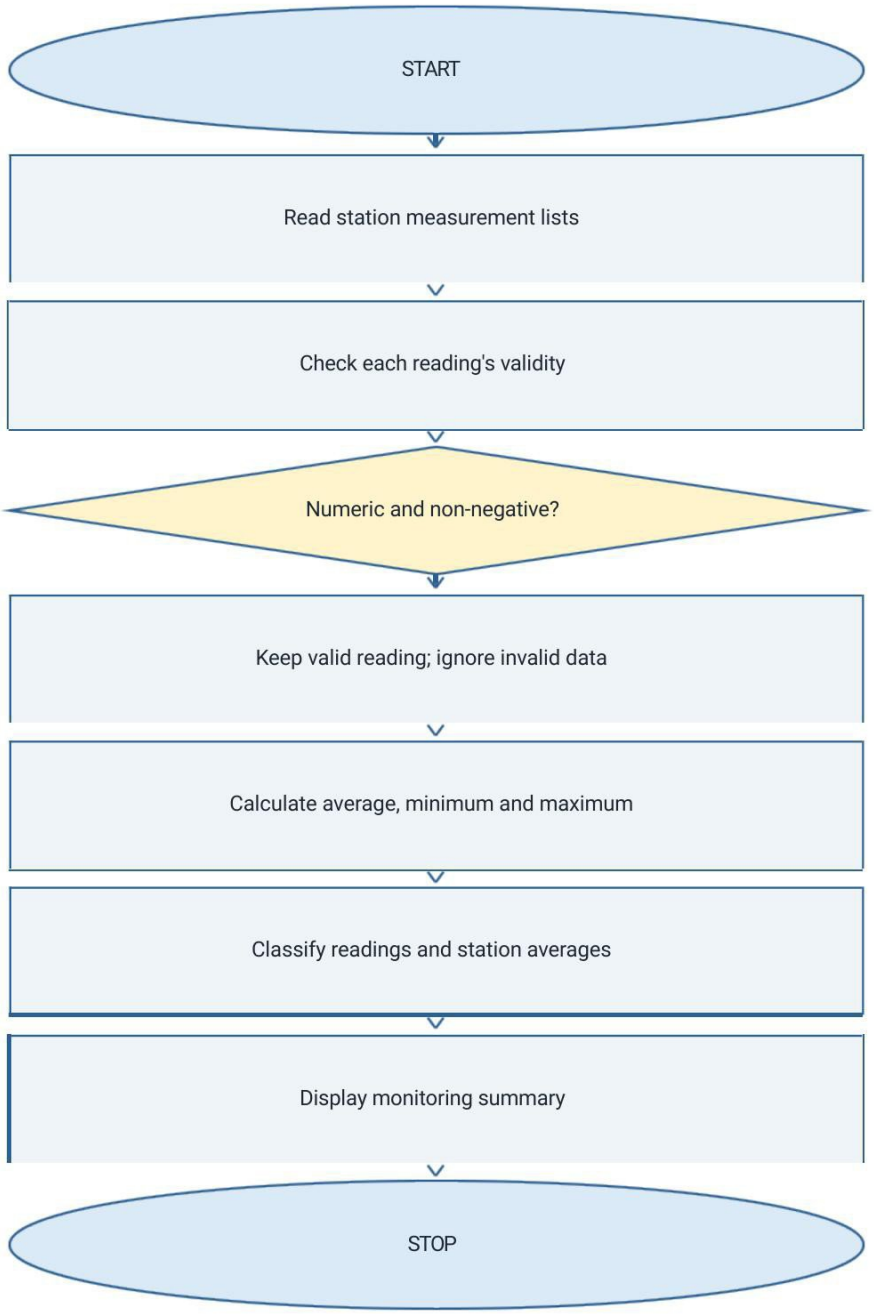
```
Station: Station A
Skipped invalid readings: 2
Valid readings: [10, 20, 30]
Average: 20.0 - Good
Minimum: 10
Maximum: 30
Categories: ['Good', 'Good', 'Good']
Station: Station C
Skipped invalid readings: 0
No valid readings
```

10. Conclusion

The program cleans station readings, calculates statistics and classifies results. Changing the threshold list allows the same logic to be reused for other environmental measurements.

PROJECT 6: ENVIRONMENTAL MONITORING READINGS

Flowchart



PROJECT 7: MOBILE PLAN COMPARISON

1. Problem Statement

Mobile plans have different prices, validity periods, data limits and call minutes. Excess usage may make a low-priced plan more expensive overall.

2. Objectives

- Store plans as tuples.
- Calculate excess data and call charges.
- Check validity and budget.
- Recommend the lowest-cost eligible plan.

3. Assumptions

- A plan stores name, price, validity days, data GB, call minutes, excess charge per GB and per minute.
- Total cost is price plus excess charges.
- Eligibility requires enough validity and total cost within budget.
- Ties go to the lower base price.
- Taxes and SMS are ignored.

4. Algorithm

- Read plans and user needs.
- Calculate excess data and minutes.
- Calculate total cost.
- Reject plans with short validity or over-budget totals.
- Choose the lowest-cost eligible plan.
- Print comparison and recommendation.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
extra_data = max(0, usage_data - plan_data)
extra_minutes = max(0, usage_minutes - plan_minutes)
total = price + extra_data*rate_per_gb + extra_minutes*rate_per_min
eligible if validity >= days_needed and total <= budget
choose eligible plan by (total cost, base price)
```

7. Python Source Code

```
plans = [
    ("Basic", 199, 28, 20, 100, 10, 1),
    ("Value", 299, 28, 40, 200, 8, 1),
    ("Max", 499, 56, 100, 500, 5, 1)
]
usage = {"data": 40, "minutes": 150, "days": 28}
budget = 450
results = []
for name, price, validity, data, minutes, rate_gb, rate_min in plans:
    extra_data = max(0, usage["data"] - data)
    extra_minutes = max(0, usage["minutes"] - minutes)
    extra = extra_data * rate_gb + extra_minutes * rate_min
    total = price + extra
    if validity < usage["days"]:
        status = "Validity too short"
```

```
    elif total > budget:
        status = "Over budget"
    else:
        status = "Eligible"
        results.append((total, price, name))
    print(name, "Base:", price, "Extra:", extra, "Total:", total, status)
if results:
    best = min(results)
    print("Best plan:", best[2], "Total cost:", best[0])
else:
    print("No plan fits the requirements.")
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Usage within plan allowance	Excess charges 0
2	Basic: use 30 GB and 120 minutes	Excess charge I40
3	Need 30 days; plan valid 28 days	Not eligible: validity too short
4	Budget I100	No eligible plan
5	Usage 40 GB, 150 minutes, 28 days; budget I450	Value plan recommended

9. Sample Output

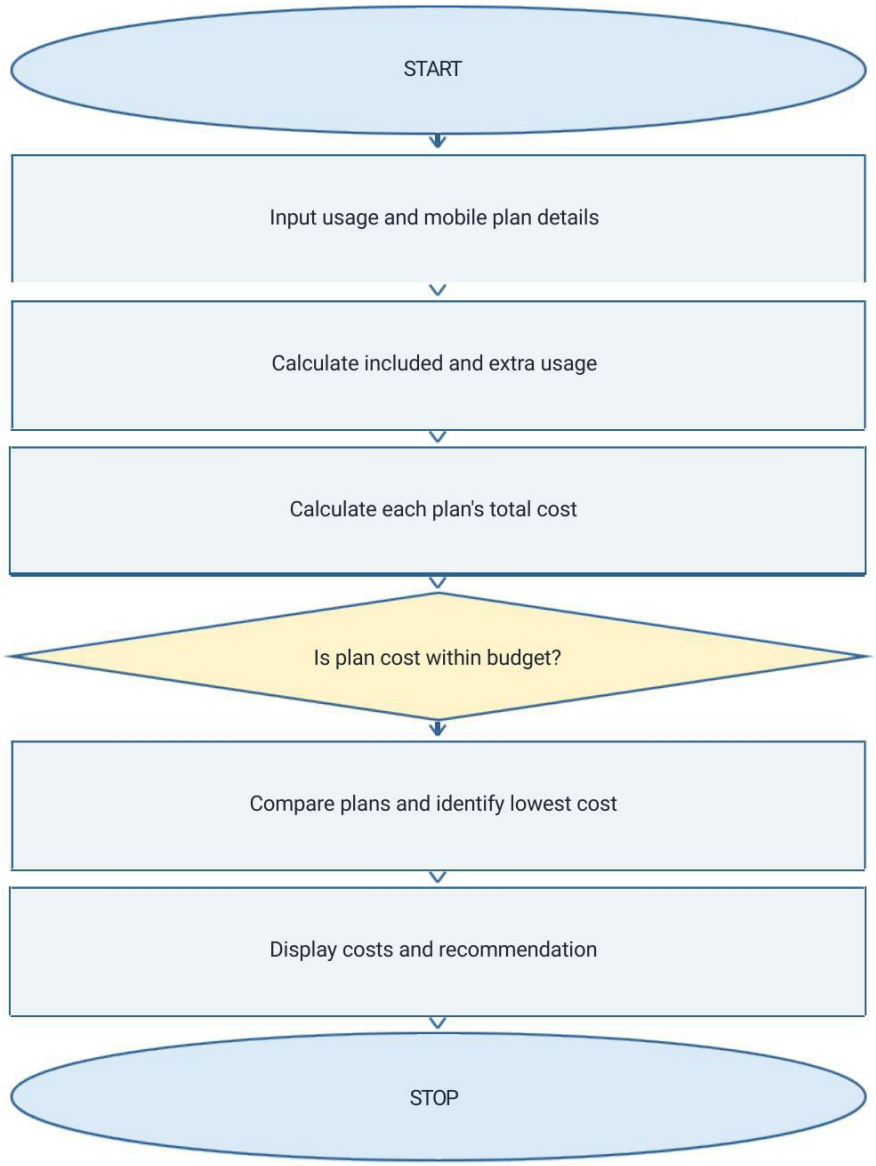
```
Basic Base: 199 Extra: 201 Total: 400 Eligible
Value Base: 299 Extra: 0 Total: 299 Eligible
Max Base: 499 Extra: 0 Total: 499 Over budget
Best plan: Value Total cost: 299
```

10. Conclusion

The program compares the full cost of mobile plans, including excess charges, validity and budget. It recommends the lowest-cost plan that meets the user's needs.

PROJECT 7: MOBILE PLAN COMPARISON

Flowchart



PROJECT 8: PUBLIC TRANSPORT REVENUE RECONCILIATION

1. Problem Statement

A bus or metro service needs to compare expected ticket revenue with actual collections for each route and identify shortages, excesses and invalid ticket records.

2. Objectives

- Store fares and category multipliers in dictionaries.
- Calculate expected route revenue.
- Compare expected and collected totals.
- Flag MATCH, SHORTAGE or EXCESS.
- List invalid tickets.

3. Assumptions

- Base fares: R1 I20, R2 I35, R3 I50.
- Multipliers: Adult 1.0, Student 0.5, Senior 0.6, Child 0.0.
- Difference within I1 counts as a match.
- Unknown route/category and negative passenger counts are invalid.

4. Algorithm

- Read ticket records.
- Validate route, category and count.
- Calculate each ticket fare.
- Add valid ticket revenue by route.
- Compare with collected revenue and classify difference.
- Print report and invalid records.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
fare = base_fare[route] * category_multiplier[category]
expected[route] += fare * passenger_count
difference = collected - expected
if abs(difference) <= 1: MATCH
elif difference < 0: SHORTAGE
else: EXCESS
```

7. Python Source Code

```
base_fares = {"R1": 20, "R2": 35, "R3": 50}
multipliers = {"Adult": 1.0, "Student": 0.5, "Senior": 0.6, "Child": 0.0}
tolerance = 1
tickets = [
    ("R1", "Adult", 2),
    ("R1", "Student", 2),
    ("R2", "Senior", 3),
    ("R9", "Adult", 1),
    ("R3", "Tourist", 2),
    ("R2", "Child", 4)
]
collected = {"R1": 60, "R2": 70, "R3": 10}
```

```
expected = {route: 0.0 for route in base_fares}
invalid = []

for route, category, count in tickets:
    if route not in base_fares or category not in multipliers or count < 0:
        invalid.append((route, category, count))
        continue
    expected[route] += base_fares[route] * multipliers[category] * count

print("Route Expected Collected Difference Status")
for route in base_fares:
    actual = collected.get(route, 0)
    diff = actual - expected[route]
    if abs(diff) <= tolerance:
        status = "MATCH"
    elif diff < 0:
        status = "SHORTAGE"
    else:
        status = "EXCESS"
    print(route, expected[route], actual, diff, status)

print("Invalid tickets:", invalid)
```

8. Test Cases

Test	Input / Condition	Expected Result
1	R1, Adult	Fare I20
2	R2, Student	Fare I17.50
3	R3, Senior	Fare I30
4	R9, Adult	Invalid ticket
5	Expected I100, collected I80	SHORTAGE I20
6	Expected I100, collected I130	EXCESS I30

9. Sample Output

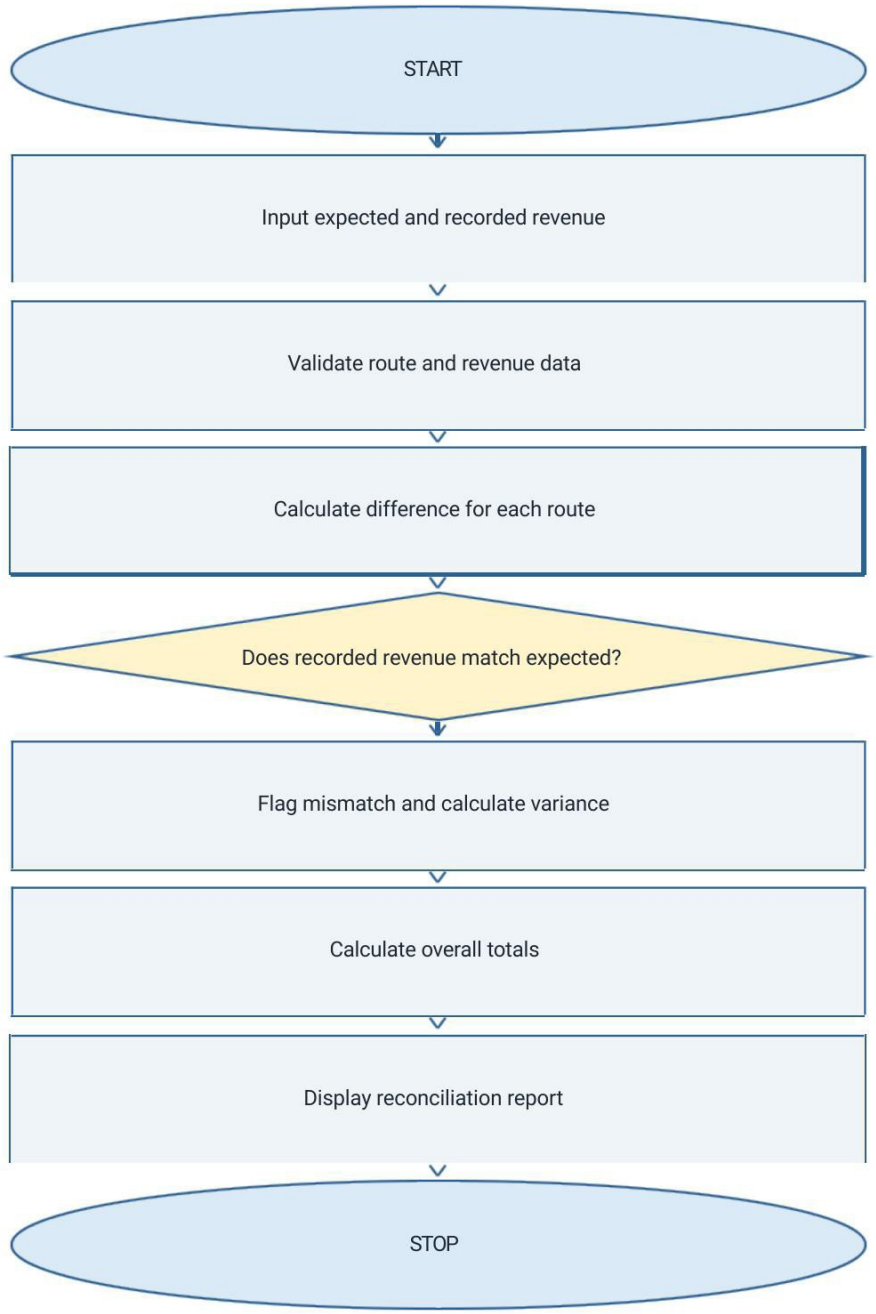
```
Route Expected Collected Difference Status
R1   60.0  60    0.0    MATCH
R2   63.0  70    7.0    EXCESS
R3   0.0   10    10.0   EXCESS
Invalid tickets: [('R9', 'Adult', 1), ('R3', 'Tourist', 2)]
```

10. Conclusion

The system calculates expected revenue from valid tickets, compares it with actual collections and flags route-level differences. Invalid ticket records are listed separately for review.

PROJECT 8: PUBLIC TRANSPORT REVENUE RECONCILIATION

Flowchart



PROJECT 9: HOSPITAL BILLING AND CONCESSIONS

1. Problem Statement

A hospital calculates consultation, diagnostic and treatment charges. Patient data must be validated, the highest eligible concession applied, and the net bill assigned a payment category.

2. Objectives

- Validate patient records.
- Calculate subtotal and concession.
- Apply the concession cap.
- Classify the net amount into a payment category.
- Print valid bills and invalid-record errors.

3. Assumptions

- Senior age is 60 or more: 10% discount.
- Low-income patient: 20% discount.
- Subtotal above I20,000: 5% discount.
- Discounts do not stack; use the highest applicable percentage.
- Maximum concession I5,000.
- Category A up to I2,000; B up to I10,000; C above I10,000.
- Name required, age 0–120, charges non-negative, total not zero.

4. Algorithm

- Validate each record.
- Calculate subtotal.
- Find highest eligible discount.
- Cap the concession at I5,000.
- Calculate net bill.
- Assign category and payment rule.
- Print bill or validation errors.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
validate name, age and charges
subtotal = consultation + diagnostic + treatment
percent = maximum applicable concession
discount = min(subtotal * percent / 100, 5000)
net = subtotal - discount
if net <= 2000: Category A
elif net <= 10000: Category B
else: Category C
```

7. Python Source Code

```
def process_patient(record):
    if len(record) != 7:
        return "INVALID: wrong number of fields"
    patient_id, name, age, low_income, consultation, diagnostic, treatment = record
    errors = []
    if not name.strip():
```

```
            errors.append("Name is missing")
        if not isinstance(age, int) or not 0 <= age <= 120:
            errors.append("Age must be between 0 and 120")
        charges = [consultation, diagnostic, treatment]
        if any(not isinstance(x, (int, float)) or x < 0 for x in charges):
            errors.append("Charges must be non-negative")
        if errors:
            return "INVALID: " + "; ".join(errors)
        subtotal = sum(charges)
        if subtotal == 0:
            return "INVALID: Total bill is zero"
        discounts = []
        if age >= 60:
            discounts.append(10)
        if low_income:
            discounts.append(20)
        if subtotal > 20000:
            discounts.append(5)
        percent = max(discounts, default=0)
        concession = min(subtotal * percent / 100, 5000)
        net = subtotal - concession
        if net <= 2000:
            category, rule = "A", "Pay in full at counter"
        elif net <= 10000:
            category, rule = "B", "Pay in full or in 2 installments"
        else:
            category, rule = "C", "50% advance plus installment plan"
        return {
            "patient": name, "subtotal": subtotal, "discount_percent": percent,
            "concession": concession, "net": net, "category": category, "rule": rule
        }
    patients = [
        (1, "Ravi", 35, False, 700, 500, 500),
        (2, "Sita", 68, False, 3000, 2100, 2000),
        (3, "Mohan", 45, True, 5000, 4000, 6500),
        (4, "", 30, False, 100, 200, 300),
        (5, "Kiran", 40, False, 1000, -300, 200)
    ]
    for patient in patients:
        print(process_patient(patient))
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Age 35, bill I1,700, no concession	Net I1,700, Category A
2	Senior, bill I7,100	10% concession I710, net I6,390, Category B
3	Low-income, bill I15,500	20% concession I3,100, net I12,400, Category C
4	Name is ""	Invalid: name missing
5	Diagnostic charge −I300	Invalid: charge must be non-negative

9. Sample Output

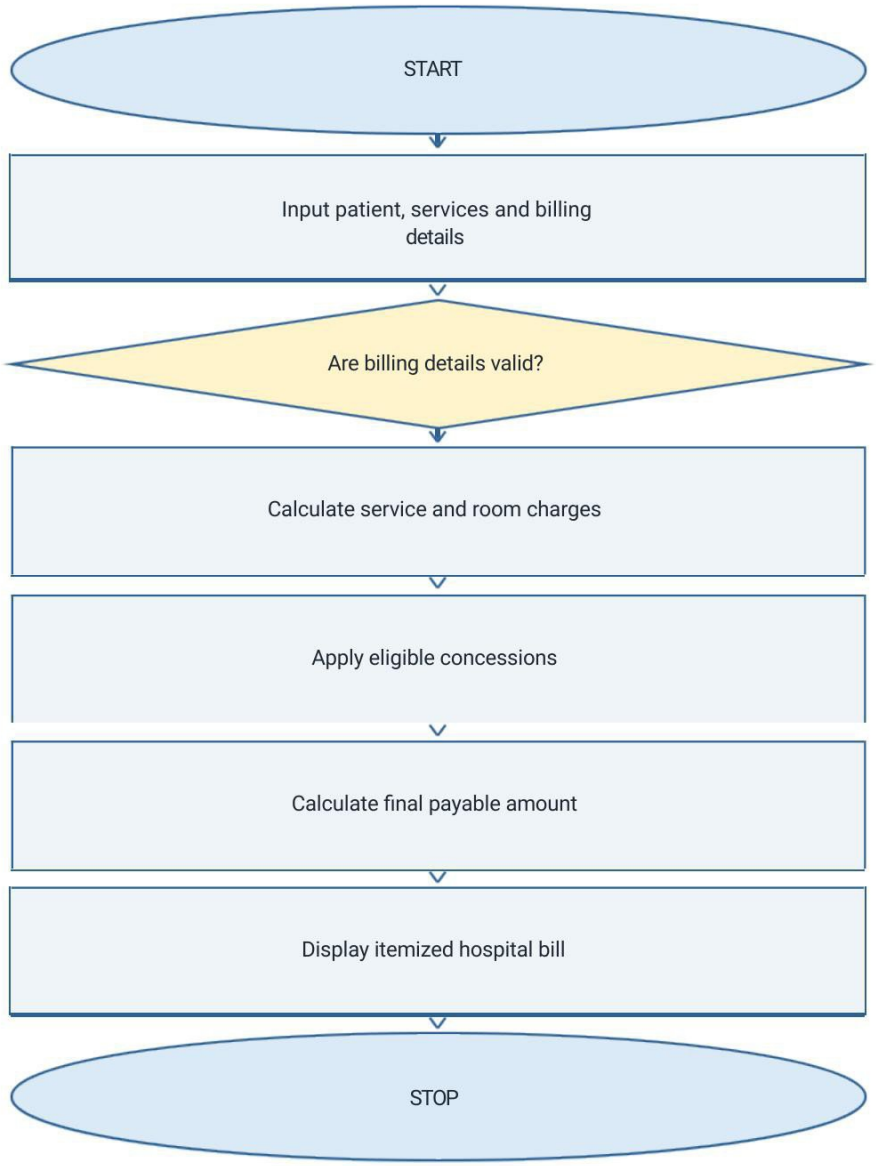
```
Ravi: subtotal 1700, concession 0, net 1700, Category A
Sita: subtotal 7100, concession 710, net 6390, Category B
Mohan: subtotal 15500, concession 3100, net 12400, Category C
INVALID: Name is missing
INVALID: Charges must be non-negative
```

10. Conclusion

The billing system validates records, calculates the highest eligible concession within the cap, and assigns payment categories based on the net bill.

PROJECT 9: HOSPITAL BILLING AND CONCESSIONS

Flowchart



PROJECT 10: WAREHOUSE INVENTORY RECONCILIATION

1. Problem Statement

A warehouse compares recorded stock movements with actual closing stock. Discrepancies must be flagged and products below reorder level must be replenished.

2. Objectives

- Validate stock records.
- Calculate expected closing stock.
- Compare expected and actual stock.
- Flag OK, SHORTAGE or SURPLUS.
- Calculate replenishment quantities.
- List invalid records.

3. Assumptions

- Record fields: product, opening, incoming, dispatched, actual closing, reorder level, max level.
- Quantities are non-negative whole numbers.
- Expected closing = opening + incoming − dispatched.
- Dispatch cannot exceed available stock.
- Maximum level cannot be below reorder level.
- Reorder to maximum level if actual stock is below reorder level.

4. Algorithm

- Read product records.
- Validate field count and quantities.
- Reject impossible records.
- Calculate expected closing and difference.
- Assign status using tolerance.
- Calculate order quantity.
- Print report and invalid records.

5. Flowchart

See detailed flowchart on the following page

6. Pseudocode

```
expected = opening + incoming - dispatched
difference = actual_closing - expected
if abs(difference) <= tolerance: OK
elif difference < 0: SHORTAGE
else: SURPLUS
if actual_closing < reorder_level:
    order = max_level - actual_closing
else:
    order = 0
```

7. Python Source Code

```
TOLERANCE = 0
def reconcile(record):
    if len(record) != 7:
        return {"error": "Wrong number of fields"}
    product, opening, incoming, dispatched, actual, reorder, maximum = record
```

```
    quantities = [opening, incoming, dispatched, actual, reorder, maximum]
    if any(not isinstance(q, int) or q < 0 for q in quantities):
        return {"product": product, "error": "Quantities must be whole numbers >= 0"}
    if dispatched > opening + incoming:
        return {"product": product, "error": "Dispatched is more than available stock"}
    if maximum < reorder:
        return {"product": product, "error": "Max level is less than reorder level"}
    expected = opening + incoming - dispatched
    difference = actual - expected
    if abs(difference) <= TOLERANCE:
        status = "OK"
    elif difference < 0:
        status = "SHORTAGE"
    else:
        status = "SURPLUS"
    order = maximum - actual if actual < reorder else 0
    return {
        "product": product, "expected": expected, "actual": actual,
        "difference": difference, "status": status, "reorder": order
    }
records = [
    ("Rice", 500, 200, 450, 250, 100, 400),
    ("Oil", 300, 100, 200, 190, 100, 550),
    ("Sugar", 200, 50, 100, 160, 80, 300),
    ("Salt", 10, 5, 20, 0, 10, 50),
    ("Tea", 100, 0, 20, -5, 10, 100)
]
print("Product Expected Actual Difference Status Reorder")
for record in records:
    result = reconcile(record)
    if "error" in result:
        print(result.get("product", "Unknown"), "INVALID:", result["error"])
    else:
        print(result["product"], result["expected"], result["actual"],
              result["difference"], result["status"], result["reorder"])
```

8. Test Cases

Test	Input / Condition	Expected Result
1	Opening 500, incoming 200, dispatched 450	Expected closing 250
2	Expected 250, actual 240	Difference −10, SHORTAGE
3	Expected 250, actual 260	Difference +10, SURPLUS
4	Actual 40, reorder 100, max 400	Order 360
5	Dispatched 20 but only 15 available	Invalid record

9. Sample Output

```
Product Expected Actual Difference Status Reorder
Rice 250 250 0 OK 0
Oil 200 190 -10 SHORTAGE 360
Sugar 150 160 10 SURPLUS 140
Salt INVALID: Dispatched is more than available stock
Tea INVALID: Quantities must be whole numbers >= 0
```

10. Conclusion

The program reconciles stock movements, identifies shortages and surpluses, rejects invalid data and calculates replenishment quantities. This helps a warehouse maintain accurate inventory records.

PROJECT 10: WAREHOUSE INVENTORY RECONCILIATION

Flowchart

